

Abstract Factory

Create families of related objects through one interface — without ever naming their concrete classes.

 DIVE 01 · 10 km
Airplane porthole
The big picture & contours

→

 DIVE 02 · 1 km
Train window
Theses, examples, terms

→

 DIVE 03 · 100 m
Park walk
How the pieces relate

→

 DIVE 04 · 1 cm
Microscope
Exact wording & gotchas

 Airplane porthole

DIVE 01 · ORIENTATION

▼ 10 km

A factory of factories — one object that mints a whole family of related products guaranteed to fit together.

THE CONTOURS

- Families, not single objects

you get matching sets, never mismatched parts

- Swap the family at one point

pick the concrete factory once, at startup

- Easy to extend by family

— but adding a product type touches every factory

USE IT WHEN — GoF applicability

 Independent of creation

a system shouldn't know how its products are made, composed or represented

 One of several families

must be configured at runtime, and products are meant to be used together

Creational family: Factory Method · Abstract Factory · Builder · Prototype · Singleton

— Abstract Factory answers “which whole family?”, and is built from many Factory Methods.

↓ DIVE DEEPER

 Train window

DIVE 02 · KEY OBJECTS

▼ 1 km

THE THREE PARTS

AbstractFactory

declares createProductA() / createProductB() for one family

ConcreteFactory

one per variant — builds products that belong together

AbstractProduct

the interfaces the client codes to; concrete classes stay hidden

IN THE WILD

 GUI toolkit themes

WinFactory vs MacFactory build matching widgets

 DocumentBuilderFactory

newInstance() yields a parser's whole object family

 java.awt.Toolkit

getDefaultToolkit() — platform peers as one family

 Cloud SDK clients

one provider factory → matching service clients

<> Client → AbstractFactory.createProductA / B() → matching ConcreteProducts of a single family.

GoF intent: “Provide an interface for creating families of related or dependent objects without specifying their concrete classes.”

↓ DIVE DEEPER

 Park walk

DIVE 03 · CONNECTIONS

▼ 100 m

Client

needs a family

→

AbstractFactory

create entry

→

which ConcreteFactory?

chosen once at config

→

✓ matching family

products that fit

WHY IT FOLLOWS

→ One factory object ⇒ every product it makes belongs to the same family

→ Program to interfaces ⇒ the client never names a concrete product or factory

→ Swap the factory ⇒ the whole product set changes in a single assignment

🔗 Built from Factory Methods (or Prototype to avoid subclassing); the concrete factory is often a Singleton; complex products may use Builder.

STRUCTURE · UML

AbstractFactory

«interface»

+ createProductA(): ProductA

+ createProductB(): ProductB

ConcreteFactory1 / 2 implement these; each returns products of one matching family. The client holds an AbstractFactory, never a concrete one.

WHAT IT BUYS YOU

- Isolates concrete classes — product class names live only inside the factories
- Swaps families easily — change the concrete factory once → the whole set switches
- Enforces consistency — products from one factory are guaranteed to match
- Hard to add a product — a new product kind forces every factory interface to change
- Usually a Singleton — one concrete factory per app is typically enough

↓ DIVE DEEPER

 Microscope

DIVE 04 · DETAILS

▼ 1 cm

Definition: “Provide an interface for creating families of related or dependent objects without specifying their concrete classes.”

— GoF, “Design Patterns”, 1994

THE CODE

```
interface GUIFactory {
  createButton(): Button;
  createCheckbox(): Checkbox;
}

class MacFactory implements GUIFactory {
  createButton() { return new MacButton(); }
  createCheckbox() { return new MacCheckbox(); }
}

// one switch picks the whole family:
const ui: GUIFactory =
  isMac ? new MacFactory() : new WinFactory();
```

VARIANTS

Factory-Method based

each create*() is a Factory Method — the classic GoF form

Parameterized

one create(kind) instead of many methods — fewer classes

DI container

the container is the factory; a module picks the family

Prototype based

the factory clones registered prototypes instead of new

Functional

a record of factory functions — no class hierarchy (JS/TS)

Kit / toolkit

GoF's original name — a “kit” of matching parts

COMPARE THE ALTERNATIVES

Property	Abstract F.	Factory M.	Builder	Prototype
Creates a whole family	✓	✗	✗	~
Hides concrete class	✓	✓	✓	✓
Add new variant easily	✓	✓	✓	✓
Add new product easily	✗	~	~	✓
Composition over inherit.	✓	✗	✓	✓

Best fit → Abstract Factory: swappable product families · Factory Method: one product via subclass · Builder: stepwise build · Prototype: clone. (*~* = partial.)

INTERVIEW PITFALLS

Q “Abstract Factory vs Factory Method?”

Factory Method makes one product, chosen by subclassing. Abstract Factory makes a family, chosen by composing/ swapping a factory object.

Q “Why is adding a product hard?”

A new product method must be added to the abstract factory and to every concrete factory — it breaks them all at once.

Q “Is the factory a Singleton?”

Usually — one concrete factory per app, built at startup. But that's a separate decision, not required.

Q “How are matching products guaranteed?”

A single concrete factory makes every product, so they all come from the same family — no mismatched pairs.

NUANCES & GOTCHAS

- Two axes of change — adding a family is easy (one new factory); adding a product is hard (touch every factory)
- vs Factory Method — Abstract Factory makes many related products via composition; Factory Method makes one via inheritance
- Consistency is the point — you can't accidentally pair a MacButton with a WinCheckbox
- Factory is often a Singleton — usually one per app, created once at configuration time
- Open/Closed asymmetry — open for new families, effectively closed for new product kinds
- JS reality — a plain object of factory functions usually beats a class hierarchy

WATCH OUT

 Anti-pattern: Overkill with only one product family or none that varies — a single Factory Method, or plain new, is simpler. Don't add the indirection until a second family is real.

 Modern take: in TS/JS reach for an object of factory functions (or a DI container) keyed by environment; you rarely need the full class hierarchy.

• VasyI Krupka · Senior Fullstack Engineer · learn-by-diving series

Source: GoF “Design Patterns” (1994) v1